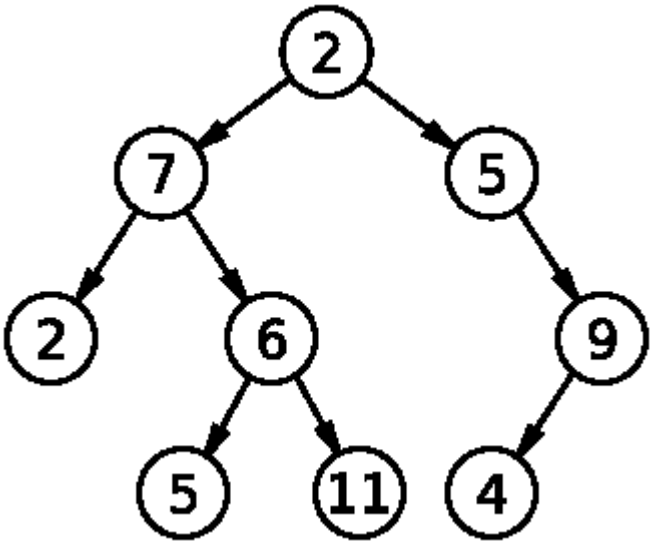
Структура данных «Дерево» и её реализация в C#

Введение

В предыдущих разделах рассматривались линейные структуры данных (массивы, списки, очереди), характеризующиеся последовательной организацией элементов. Однако значительная часть реальных информационных моделей (иерархия файловой системы, структура организации, синтаксическое дерево программы) имеет нелинейную, иерархическую природу. Для эффективного представления и обработки таких данных используется структура, называемая **деревом**.

Деревья обеспечивают компромисс между скоростью доступа к элементам (превосходя линейные списки) и возможностью динамического изменения размера (превосходя массивы). Более того, на основе деревьев строятся сбалансированные структуры, гарантирующие логарифмическую сложность основных операций поиска, вставки и удаления.



1. Теоретические основы: Определение и терминология

Дерево (Tree) — это рекурсивная структура данных, представляющая собой совокупность элементов, называемых **узлами (nodes)**, и связей между ними, называемых **ребрами (edges)**. Дерево всегда имеет единственный выделенный узел — **корень (root)**.

Формальное определение для бинарного дерева:
Бинарное дерево — это конечное множество узлов, которое либо пусто, либо состоит из корня и двух непересекающихся бинарных деревьев — левого и правого поддеревьев.

Ключевые понятия:

- **Корень (Root):** Узел верхнего уровня, не имеющий родителя.
- **Родитель (Parent):** Узел, имеющий исходящие связи к другим узлам.
- **Потомок (Child):** Узел, на который имеется ссылка от другого узла (родителя).
- **Лист (Leaf):** Узел, не имеющий потомков (терминальный узел).
- **Внутренний узел (Internal node):** Узел, имеющий хотя бы одного потомка.
- **Высота дерева (Height):** Количество рёбер на самом длинном пути от корня до какого-либо листа. Пустое дерево имеет высоту -1, дерево из одного корня — высоту 0.
- **Глубина узла (Depth):** Количество рёбер от корня до данного узла.
- **Степень узла (Degree):** Количество поддеревьев, принадлежащих узлу.

2. Абстрактный тип данных и рекурсивная природа

Ключевым свойством дерева является его **рекурсивность**. Каждый узел можно рассматривать как корень собственного поддерева. Это свойство определяет использование рекурсивных алгоритмов при работе с деревьями.

Базовое определение узла бинарного дерева на языке C# может быть представлено следующим классом:

```csharp
public class BinaryTreeNode<T>
{
    /// <summary>
    /// Данные, хранящиеся в узле.
    /// </summary>
```

```
    public T Data { get; set; }

    /// <summary>
    /// Ссылка на левый дочерний узел.
    /// </summary>
    public BinaryTreeNode<T>? Left { get; set; }

    /// <summary>
    /// Ссылка на правый дочерний узел.
    /// </summary>
    public BinaryTreeNode<T>? Right { get; set; }

    /// <summary>
    /// Инициализирует новый экземпляр узла с указанными данными.
    /// </summary>
    /// <param name="data">Данные для хранения.</param>
    public BinaryTreeNode(T data)
    {
        Data = data;
        Left = null;
        Right = null;
    }
}
```

Для решения некоторых задач (например, навигация вверх по дереву или реализация итераторов) может потребоваться наличие ссылки на родительский узел, что расширяет базовую структуру.

3. Бинарное дерево поиска (Binary Search Tree, BST)

Наиболее практически значимой разновидностью бинарного дерева является **бинарное дерево поиска**. Оно накладывает дополнительное ограничение на порядок хранения значений, что и обеспечивает эффективность поиска.

3.1. Инвариант BST

Для любого узла `N` бинарного дерева поиска должно выполняться следующее условие:

1. Все значения в левом поддереве узла `N` **меньше** значения самого узла `N`.
2. Все значения в правом поддереве узла `N` **больше** значения самого узла `N`.

   Примечание: Допускаются вариации относительно обработки дубликатов. Обычно дубликаты либо не допускаются, либо вставляются в правое поддерево.

3.2. Операция вставки (Insert)

Алгоритм вставки рекурсивно сравнивает вставляемое значение с текущим узлом и движется влево или вправо до нахождения подходящей пустой позиции.

```csharp
public class BinarySearchTree
{
    private BinaryTreeNode<int>? _root;

    public void Insert(int value)
    {
        _root = InsertRecursive(_root, value);
    }

    private BinaryTreeNode<int> InsertRecursive(BinaryTreeNode<int>? node, int value)
    {
        // Базовый случай: найдено пустое место
        if (node == null)
        {
            return new BinaryTreeNode<int>(value);
        }

        // Рекурсивный спуск
        if (value < node.Data)
        {
            node.Left = InsertRecursive(node.Left, value);
        }
        else if (value > node.Data)
        {
            node.Right = InsertRecursive(node.Right, value);
        }
        else
        {
            // Обработка дубликата: в данной реализации дубликаты игнорируются
            // Возможно иное поведение, например, вставка в правое поддерево.
```

```csharp
        }

        return node;
    }
}
```

3.3. Операция поиска (Search / Find)

Поиск значения осуществляется по тому же принципу: сравнение с текущим узлом и направленный спуск.

```csharp
public bool Contains(int value)
{
    return ContainsRecursive(_root, value);
}

private bool ContainsRecursive(BinaryTreeNode<int>? node, int value)
{
    if (node == null)
    {
        return false;
    }

    if (value == node.Data)
    {
        return true;
    }

    return value < node.Data
        ? ContainsRecursive(node.Left, value)
        : ContainsRecursive(node.Right, value);
}
```

3.4. Операция удаления (Remove)

Удаление узла из BST является наиболее сложной операцией из-за необходимости сохранения инварианта дерева. Существует три основных сценария:

1. **Удаление листа:** Узел просто отсоединяется от родителя.

2. **Удаление узла с одним потомком:** Удаляемый узел заменяется его единственным потомком (аналог удаления элемента из связного списка).
3. **Удаление узла с двумя потомками:** Требуется замена удаляемого узла на **преемника (successor)** — наименьший узел в правом поддереве, или на **предшественника (predecessor)** — наибольший узел в левом поддереве. Затем преемник удаляется (он гарантированно попадает под первые два случая).

3.5. Поиск минимального и максимального элемента

Вследствие свойства упорядоченности, минимальный элемент всегда находится в крайнем левом узле, максимальный — в крайнем правом.

```csharp
public int? FindMin()
{
    if (_root == null) return null;
    return FindMinRecursive(_root).Data;
}

private BinaryTreeNode<int> FindMinRecursive(BinaryTreeNode<int> node)
{
    return node.Left == null ? node : FindMinRecursive(node.Left);
}
```

4. Способы обхода дерева (Tree Traversal)

Обход дерева — это процесс систематического посещения каждого узла ровно один раз. Выделяют несколько стратегий обхода, каждая из которых применяется для решения определенных задач.

4.1. Глубинный обход (Depth-First Search, DFS)

Рекурсивные алгоритмы, которые идут вглубь дерева до достижения листа, прежде чем вернуться.

- **Прямой обход (Pre-order):** `Корень -> Левое поддерево -> Правое поддерево`
  - *Применение:* Копирование структуры дерева, сериализация.
- **Центрированный обход (In-order):** `Левое поддерево -> Корень -> Правое поддерево`

- *Применение:* Для BST результатом является последовательность, отсортированная по возрастанию.
- **Обратный обход (Post-order):** `Левое поддерево -> Правое поддерево -> Корень`
- *Применение:* Удаление дерева (сначала удаляются потомки, затем родитель), подсчет размера директории.

4.2. Обход в ширину (Breadth-First Search, BFS)

Алгоритм посещает узлы по уровням: сначала корень, затем все узлы на глубине 1, затем на глубине 2 и так далее. Реализуется с использованием очереди (`Queue`), а не стека (как рекурсия). Называется также **level-order traversal**.

5. Проблема несбалансированности и самобалансирующиеся деревья

Эффективность операций BST напрямую зависит от его **высоты** (h). В лучшем случае (сбалансированное дерево) сложность составляет **O(log n)**. Однако при вставке упорядоченных данных (например, 1, 2, 3, 4) дерево вырождается в линейный список, и высота становится равной **n**, а сложность операций падает до **O(n)**.

Для решения этой проблемы были разработаны самобалансирующиеся деревья поиска.

5.1. AVL-дерево

Строго сбалансированное дерево, названное по фамилиям изобретателей (Адельсон-Вельский и Ландис).

- **Критерий баланса:** Для любого узла разница высот левого и правого поддеревьев (balance factor) не превышает 1 по модулю.
- **Механизм:** После каждой вставки или удаления производится проверка баланса и, при необходимости, выполняется одно или несколько **вращений (rotations)** — левое, правое, большое левое, большое правое — для восстановления баланса.

5.2. Красно-черное дерево (Red-Black Tree)

Более распространенный на практике тип сбалансированного дерева.

- **Критерий баланса:** Основан на цвете узлов (красный/черный) и наборе правил (корень черный, у красного узла родитель черный, одинаковая черная высота на всех путях).
- **Особенность:** Требует меньше перестроений (поворотов и перекрашиваний) при вставке, чем AVL-дерево, что делает его предпочтительным для сценариев с интенсивными операциями записи.

6. Реализация деревьев в .NET (BCL)

Платформа .NET предоставляет готовые реализации сбалансированных деревьев, что избавляет разработчика от необходимости создавать их "с нуля".

1. `SortedSet<T>`

- Пространство имен: `System.Collections.Generic`
- Реализация: Красно-черное дерево.
- Назначение: Хранение уникальных элементов в отсортированном порядке.
- Особенности: Поддерживает операции над множествами (объединение, пересечение) и получение подмножества (`GetViewBetween`).

```csharp
SortedSet<int> sortedSet = new SortedSet<int>();
sortedSet.Add(5);
sortedSet.Add(1);
sortedSet.Add(10); // Порядок хранения: 1, 5, 10
```

2. `SortedDictionary<TKey, TValue>`

- Пространство имен: `System.Collections.Generic`
- Реализация: Красно-черное дерево.
- Назначение: Хранение пар "ключ-значение", где ключи отсортированы.
- Особенности: Поиск по ключу выполняется за O(log n).

```csharp
SortedDictionary<string, int> dictionary = new SortedDictionary<string, int>();
```

```
dictionary.Add("Alice", 25);
dictionary.Add("Bob", 30); // Ключи будут автоматически отсортированы.
```

7. Области применения

Применение деревьев в информатике чрезвычайно широко:

- **Базы данных:** Индексы на основе B-деревьев и их вариаций (B+ Tree) для быстрого поиска записей.
- **Компиляторы:** Построение абстрактного синтаксического дерева (AST) в процессе синтаксического анализа кода.
- **Сетевые технологии:** Построение маршрутов (алгоритмы на графах, частным случаем которых являются деревья).
- **Игровая индустрия:** Деревья поведения (Behavior Trees) для моделирования логики неигровых персонажей.
- **Графические интерфейсы:** Иерархическая структура визуальных компонентов (Windows Forms, WPF).

Резюме

Деревья представляют собой фундаментальную нелинейную структуру данных, необходимую для эффективной работы с иерархической информацией. Бинарное дерево поиска вводит порядок, позволяющий осуществлять поиск за логарифмическое время в среднем. Проблема вырождения решается применением самобалансирующихся механизмов (AVL, красно-черные деревья). В экосистеме .NET стандартным инструментом для работы с упорядоченными коллекциями на основе деревьев являются классы `SortedSet<T>` и `SortedDictionary<TKey, TValue>`.